

Intermediate Code Generation & Control Flow

30 Essential Concept & Mechanism Questions with Concise Answers

Subject: Compiler Design & Intermediate Code Generation | **Format:** Concept Q&A Study Guide

1. CONTROL FLOW, SDTS & BACKPATCHING

Q1. When we generate Three-Address Code for control flow statements, why do we calculate labels as inherited attributes?

Answer: Labels such as *B.true*, *B.false*, and *S.next* represent jump targets determined by the enclosing statement context (e.g., *if*, *while*, or sequential ordering). Because these target locations are defined top-down by parent constructs before the body or subexpression code is fully generated, they must be passed down the parse tree as inherited attributes.

Q2. What is short-circuit (jumping) code in intermediate code generation?

Answer: In short-circuit code, boolean operators like *&&*, *||*, and *!* do not appear as explicit operators in the output code. Instead, boolean operators translate directly into conditional and unconditional jumps, where the logical value of an expression is represented by the location reached in the code sequence.

Q3. How does using a special *fall* label reduce redundant *goto* instructions in control-flow translation?

Answer: The *fall* label indicates that control should flow naturally to the very next instruction without emitting a jump. If a true or false exit matches *fall*, the translation scheme emits a jump instruction only for the non-fall exit (or no jump at all if both exits fall through), eliminating redundant *goto* instructions.

Q4. How does a compiler handle expressions that serve dual roles—altering control flow vs. computing a boolean value?

Answer: A compiler distinguishes the two roles by syntax context and applies separate code-generation methods. When an expression governs control flow (e.g., in a *while* loop), a *jump* method generates conditional jumps. When it appears in an assignment, an *rvalue* method evaluates the expression and stores *true* or *false* into a temporary variable.

Q5. What key problem does backpatching solve in one-pass code generation?

Answer: In a single pass, the target label of a jump (such as jumping past a statement body) is often unknown when the jump instruction is emitted. Backpatching leaves the target label blank, maintains lists of unresolved jumps (*truelist*, *falselist*, *nextlist*) as synthesized attributes, and fills in the proper target address once it becomes known.

2. INTERMEDIATE REPRESENTATIONS & DAGS

Q6. What is the main structural difference between a Syntax Tree and a Directed Acyclic Graph (DAG) for an expression?

Answer: A syntax tree replicates nodes for subexpressions every time they appear in the source text. In contrast, a DAG identifies common subexpressions and allows a single interior node to have multiple parent nodes, yielding a more compact representation.

Q7. How does the value-number method construct DAG nodes without duplicate entries?

Answer: The value-number method assigns an integer index (value number) to each unique node defined by its signature (*op, left, right*). By searching a hash table of existing node signatures before creating new array records, identical subexpressions reuse existing value numbers rather than duplicating nodes.

Q8. How do quadruples and triples differ in representing three-address code?

Answer: A quadruple uses four explicit fields (*op, arg1, arg2, result*), storing compiler-generated temporaries explicitly in the *result* field. A triple omits the *result* field entirely and refers to intermediate values using the array index or position of the triple that generated them.

Q9. Why are indirect triples preferred over plain triples in optimizing compilers?

Answer: Plain triples reference intermediate values by their physical instruction position, meaning moving an instruction requires updating all references to it throughout the code. Indirect triples store an array of pointers to triples, allowing optimizations to reorder instructions simply by moving pointers without altering the underlying triple objects.

Q10. What is Static Single-Assignment (SSA) form, and what is the purpose of a ϕ -function?

Answer: SSA is an intermediate representation in which every variable is assigned a unique name. When a variable receives different definitions along converging control-flow paths, a ϕ -function (phi-function) selects the value corresponding to the path that was actually executed.

3. TYPES, DECLARATIONS & MEMORY LAYOUT

Q11. What is the difference between structural equivalence and name equivalence of type expressions?

Answer: Two types are structurally equivalent if they share the same basic type or are constructed by applying identical constructors to structurally equivalent types. Name equivalence occurs when type names are treated as distinct types standing for themselves rather than as abbreviations for underlying type expressions.

Q12. Why are inherited attributes necessary when computing array types like $\text{int}[2][3]$?

Answer: In type declarations like *int*, the basic type (*int*) is parsed first at the top of the subtree, while array dimensions appear below. An inherited attribute passes the basic type down through the component chain, allowing synthesized attributes to build up the nested *array(2, array(3, integer))* type bottom-up.

Q13. What is padding in storage layout, and why do compilers use it?

Answer: Padding consists of unused bytes inserted between variables or fields to satisfy target machine alignment constraints (e.g., placing 4-byte integers at addresses divisible by 4). Proper alignment allows the target hardware to execute faster memory access operations without extra alignment instructions.

Q14. How does a compiler compute relative addresses (offsets) for a sequence of variable declarations?

Answer: The compiler uses a tracking variable (such as *offset*) initialized to 0 before processing declarations. As each variable is declared, its relative address is set to the current *offset* in the symbol table, and *offset* is incremented by the byte width of the variable's type.

Q15. How are record (or *struct*) field names isolated from outer scope variables during compilation?

Answer: When entering a record declaration, the compiler pushes the current symbol table and *offset* onto a stack and creates a fresh symbol table for the record fields. Offsets for record fields are computed relative to the start of the record's own data area, preventing name conflicts with variables outside the record.

4. TRANSLATION OF EXPRESSIONS & ARRAYS**Q16. How does three-address code simplify the compilation of complex arithmetic expressions?**

Answer: Three-address code breaks down multi-operator expressions into a linear sequence of simple instructions containing at most one operator per statement. Intermediate results from subexpressions are computed sequentially and held in compiler-generated temporary variables.

Q17. What is the address calculation formula for a 1D array reference $A[i]$ with lower bound *low* and element width *w*?

Answer: The relative address is computed as $\text{base} + (i - \text{low}) \times w$, which can be rewritten as $i \times w + c$, where $c = \text{base} - \text{low} \times w$. The subexpression *c* represents a constant offset that can be precalculated at compile time if array bounds are fixed.

Q18. What is the difference between row-major and column-major array storage order?

Answer: In row-major layout (used in C and Java), multi-dimensional arrays are stored row by row, causing the rightmost subscript to vary fastest in contiguous memory. In column-major layout (used in Fortran), arrays are stored column by column, causing the leftmost subscript to vary fastest.

Q19. Why can address constants not be precalculated at compile time for dynamic arrays?

Answer: Compile-time constant precalculation relies on knowing the upper and lower array bounds during compilation. Because dynamic array bounds are only known when the program executes, address calculation formulas must be fully evaluated at run time.

Q20. How do symbolic type widths help maintain machine independence in intermediate code?

Answer: Instead of hardcoding concrete byte sizes (such as 4 bytes for an integer), intermediate code can represent widths using symbolic constants. This allows the target code generator to plug in machine-specific word sizes without modifying the front-end intermediate translation rules.

5. RUN-TIME ENVIRONMENTS & STACK ALLOCATION**Q21. Why is a run-time stack appropriate for managing storage for procedure activations?**

Answer: Procedure activations nest cleanly in time—meaning a called procedure must complete before its caller can finish. A control stack pushes local storage when a procedure is called and pops it upon return, allowing non-overlapping procedure calls to share the same memory space.

Q22. What is an activation tree, and how do procedure calls and returns relate to it?

Answer: An activation tree depicts the execution history of procedure activations, where each node represents an activation and children represent called procedures. The sequence of procedure calls corresponds to a preorder traversal of the tree, while returns correspond to a postorder traversal.

Q23. What are the key fields contained within a standard activation record (stack frame)?

Answer: An activation record typically contains actual parameters, return values, control links, access links, saved machine status (such as return addresses and register values), local data, and temporaries.

Q24. What is the function of the control link in an activation record?

Answer: The control link is a pointer stored inside the activation record that points directly to the activation record of the calling procedure. It allows the run-time system to restore the caller's stack frame when the called procedure terminates.

Q25. How are variable-length local arrays allocated on the run-time stack?

Answer: Fixed-length fields and status pointers are placed at known, fixed offsets relative to the top-of-stack pointer. Variable-length local arrays are allocated on the stack past the fixed fields, and accessed dynamically using positive offsets from the top-of-stack pointer at run time.

6. HANDLING OBJECT-ORIENTED LANGUAGE FEATURES**Q26. How are field offsets maintained when a subclass inherits fields from a superclass?**

Answer: The compiler allocates a contiguous data block for class instances where inherited fields retain the exact relative addresses assigned in the superclass memory layout. Fields unique to the subclass are appended immediately after the inherited fields.

Q27. What happens to visibility modifiers (*public*, *private*, *protected*) during intermediate code generation?

Answer: Visibility modifiers are design-level constraints checked purely during semantic analysis. Once access permissions are validated, visibility keywords are completely stripped, and intermediate code is generated without any visibility enforcement.

Q28. How does a compiler translate a class member function call like *obj.method(arg)* into machine-executable form?

Answer: The compiler converts the member function into a global function and inserts the object instance (*this*) as an explicit first parameter, transforming *obj.method(arg)* into *method(obj, arg)*.

Q29. What is a virtual method table (vtable), and why is it required for dynamic method dispatch?

Answer: A vtable is a compiler-generated table of function pointers for a class, mapping method names to their specific implementations. It is required because a variable's actual object type may vary at run time, requiring an indirect lookup through the vtable to execute the correct overridden method.

Q30. What is the purpose of the *vpointer* field added to class instances?

Answer: The compiler appends a hidden *vpointer* field to every object instance, pointing directly to its class's vtable. During a virtual method call, the compiler follows *vpointer* to fetch the vtable and dynamically resolve the function address at run time.